
ICCAD 2026 CAD Contest Problem D

Timing Fixing by Useful Skew

Aaron Wu

Department of Electrical Engineering
National Taiwan University
b13901011@ntu.edu.tw

Yucheng Liu

Department of Electrical Engineering
National Taiwan University
b13901104@ntu.edu.tw

Sheng-Yu Cheng

Department of Electrical Engineering
National Taiwan University
b13901088@ntu.edu.tw

Hong-Bin Lai

Department of Electrical Engineering
National Taiwan University
b13901078@ntu.edu.tw

Abstract

1 Introduction

ICCAD Contest 2026 Problem D asks us to improve such violations by modifying the clock tree rather than changing the data-path logic. The available operations are local clock-tree edits: inserting a new buffer, removing an inserted buffer, or resizing an existing buffer. These operations change clock arrival times at flip-flops and therefore change the skew between launch and capture registers.

The key opportunity is useful skew. A nonzero clock skew is not always harmful: delaying the capture clock can give a setup-critical data path more time, while delaying the launch clock can help avoid hold violations by postponing data launch. However, each clock-tree edit may affect many data paths at once. Improving one endpoint may degrade another path, and adding buffers also increases area. The optimization problem is therefore not only to remove timing violations, but to balance setup improvement, hold improvement, and area cost.

In this project, we model the clock tree as a mutable state and search over a space of legal buffer insertion, removal, and resizing operations. Our solver uses candidate policies to decide which moves should be evaluated, and acceptance policies to decide which evaluated move should be applied. This framework allows us to compare greedy search, simulated annealing, iterated simulated annealing, two-step timing repair, and tabu search under a common clock-tree timing model. The experimental discussion focuses on how these policies trade off setup slack, hold slack, and area.

2 Problem Formulation

2.1 Slack, Skew, and Score

A data path connects a launch flip-flop to a capture flip-flop through combinational logic. Let D be the data-path delay, T_{clk} be the clock period, T_{setup} be the setup requirement, and T_{hold} be the hold requirement. Let C_{launch} and C_{capture} denote the clock arrival times from the clock source to the launch and capture flip-flops. We define clock skew as

$$\text{skew} = C_{\text{capture}} - C_{\text{launch}}. \quad (1)$$

With this convention, the setup and hold slacks used in the presentation are

$$\text{setup_slack} = T_{\text{clk}} - T_{\text{setup}} - D + \text{skew}, \quad (2)$$

$$\text{hold_slack} = D - T_{\text{hold}} - \text{skew}. \quad (3)$$

A negative setup slack means that data arrives too late at the capture flip-flop. A negative hold slack means that data arrives too early and may overwrite the previous value. Therefore, a setup violation can often be improved by increasing the capture clock delay, while a hold violation can often be improved by increasing the launch clock delay.

The objective combines timing and area. Timing quality is measured by total negative slack (TNS) and worst negative slack (WNS) under the setup-oriented SS corner and the hold-oriented FF corner. Area is the total area of the clock-tree

buffers. Given a baseline metric value x_0 and a final value x , we use the relative improvement form

$$I(x, x_0) = 1 - \frac{x}{x_0}, \quad (4)$$

when x_0 is nonzero. This rewards reducing negative timing metrics toward zero and rewards reducing area below the baseline. The implemented score is

$$\begin{aligned} S_{\text{setup}} &= I(\text{TNS}_{SS}, \text{TNS}_{SS}^0) + I(\text{WNS}_{SS}, \text{WNS}_{SS}^0), \\ S_{\text{hold}} &= I(\text{TNS}_{FF}, \text{TNS}_{FF}^0) + I(\text{WNS}_{FF}, \text{WNS}_{FF}^0), \\ S_{\text{area}} &= 1 - \frac{\text{Area}}{\text{Area}^0}, \\ S &= \alpha S_{\text{setup}} + \beta S_{\text{hold}} + \gamma S_{\text{area}}, \\ \alpha &= 0.5, \quad \beta = 0.25, \quad \gamma = 0.25. \end{aligned} \quad (5)$$

This formula explains the main trade-off in the problem: aggressive buffer insertion may improve setup or hold slack, but the same move can reduce the area component and may also hurt timing on other paths. The actual parameter is not given; therefore, we set parameters based on the importance for evaluation.

2.2 Optimization Moves

The state space consists of all clock trees reachable from the input tree by legal edits. A state records which buffers are present, the cell type of each buffer, and the parent-child connectivity of the clock tree. Original contest nodes remain part of the design, while buffers inserted by the optimizer may be removed later.

Our optimizer uses three move types. First, an *insert* move places a new buffer on an existing clock-tree edge. This increases the clock delay of the downstream subtree and can be used to adjust useful skew, but it also increases area. Second, a *remove* move deletes a buffer that was inserted by the optimizer and reconnects the surrounding tree. This can recover area, but it may undo a previous timing repair. Third, a *resize* move changes the cell type of an existing buffer. Resizing gives a finer way to increase or decrease clock delay and area without changing the clock-tree topology.

Because the number of possible edit sequences is large, the solver cannot enumerate the full state space. Instead, each iteration generates a smaller set of candidate moves and evaluates their impact on the current timing and score. The rest of the report describes how we construct these candidate sets and how different search policies decide which move to accept.

3 Methodology

3.1 Model for Optimization

Clock Tree. The clock tree is the primary data structure in our optimization framework. It consists of the clock source as the root, buffers as internal nodes, and flip-flops as leaves. The supported operations include buffer insertion, buffer removal, and buffer resizing.

Data Path Graph. The data path graph stores the launch flip-flop, capture flip-flop, and data-path delay information. This information is required to evaluate whether a clock-tree modification improves timing slack.

Timing State. The timing state maintains timing and area metrics, including setup slack, hold slack, total negative slack (TNS), worst negative slack (WNS), and area. These metrics are updated and queried during optimization.

Design Philosophy.

Minimize Re-computation. Recomputing timing information for the entire clock tree after every modification is computationally expensive. Therefore, we employ an incremental timing cache that updates only the affected paths instead of the entire structure.

Recoverability. The optimization process must support undo operations so that a modification can be reverted efficiently when the move is rejected.

3.2 Candidate Policies

Violation Path. We check all data paths and find out those whose slack is negative and smallest, which means the data path violate timing constrain. If setup constrain is violated, the buffers near capture flip-flop is modified. Otherwise, if hold constrain is violated, the buffers near launch flip-flop is modified

Upstream Window. We look for violation endpoint (i.e. capture flip-flop) and explore several buffers toward upstream of clock tree. The policy adds more candidate for optimization, but also more time consuming.

Critical Endpoint. We define *criticality* as an evaluation metric for how much a flip-flop worsen total negative slack. Negative setup slack is added to violation path's capture flip-flop's criticality, and negative hold slack is added to launch flip-flop's criticality, respectively. We try to modify the flip-flop with largest criticality to optimize the violation on more than one data path.

Random. The policy randomly generate a set of moves from inserting buffer, removing inserted buffer, and resizing buffer with a certain probability.

3.3 Accept Policies

Greedy Approach. Based on chosen candidate policy, we apply all generated candidate operations to current clock tree and calculate $\Delta Score$. The strategy accept only operations with $\Delta Score > 0$ and the largest $\Delta Score$.

Two Step Optimization. It's similar to greedy approach but with two stage focusing on different objective. In the first stage, we try to optimize timing constrain and neglect the penalty caused by area increment. In the second stage, we try to remove and resize buffers to reduce area penalty with timing performance maintained.

Simulated Annealing (SA). The strategy randomly come up with an move from candidate pool and apply the move on current clock tree. If $\Delta Score > 0$ then the move is accepted, otherwise with a certain probability determined by the temperature, this move will be accepted. When the temperature is low, the strategy act like greedy approach.

Iterated Simulated Annealing (ISA). The strategy take turns doing standard simulated annealing step and greedy step, trying to explore more possible moves.

Tabu Search. In each step of the strategy, we pick the best move among all candidates greedily. If the move is non-tabu move, then we directly accept the move, otherwise only if the move creates new global optimal will the move be accepted.

Candidate Pool. In our design of simulated annealing and tabu search, candidate pool is *union pool* or *sampled union pool*. The union pool puts candidates generated by all candidate policies into the pool for decision. On the other hand, sampled union pool randomly choose a candidate policy first, and randomly choose a move from all possible moves generated by the candidate policy.

4 Experimental Results and Discussion

4.1 Using Different Candidate Policy

We discuss different candidate policy on greedy algorithm. **Score Analysis.** Generally, the best candidate policy is union pool, while random and upstream window follow closely behind. However, violation path and critical endpoint performs not satisfying at all. The two policies can not make a great improvement on both setup slack and hold slack, contributing to a much lower grade. On the other hand, the other three policies has almost the same performance on setup slack optimization, and the tiny difference is that union pool policy has better performance on hold slack and initial area.

Possible Reason. Greedy algorithms always choose a best current move, therefore it needs as many candidates as possible. Violation path and critical endpoint policies can not generate as many candidates as the other strategies, and therefore their performance is not satisfying.

4.2 Using Different Accept Policy

We discuss different accept policy on simulated annealing algorithms, tabu search and two-step optimization.

4.2.1 Random Candidate

The best accept policy is tabu search, while the worst is two-step optimization. To begin with, two-step optimization becomes oscillating in the last stage, and its performance on hold slack and area is much weaker compared to the other three. Tabu search performs slightly better on setup slack and hold slack, while ISA is also slightly better than SA. The most significant is area, tabu search greatly decrease area cost.

4.2.2 Union Pool and Sampled Union Pool

The worst policy is two-step optimization, and the others performs nearly identically. SA and ISA aggressively optimizes setup slack, however, the price is that they both sacrifice area performance. On the contrary, tabu search tries to optimize timing performance with almost the same area cost maintained.

4.3 Overall Performance

Table 1: Performance comparison of different optimizers.

Optimizer	Avg. Score	Setup	Hold	Area	Area Ratio	Std.
tabu-random	1.140 722	1.276 422	1.916 255	0.093 788	0.906 212	0.235 526
isa-random	1.084 773	1.322 733	1.737 303	-0.043 676	1.043 676	0.273 375
tabu-union-pool	1.062 058	1.259 221	1.731 261	-0.001 474	1.001 474	0.274 410
isa-sampled-union-pool	1.044 759	1.521 136	1.721 173	-0.584 406	1.584 406	0.183 581
sa-sampled-union-pool	1.044 015	1.653 149	1.833 949	-0.964 191	1.964 191	0.165 838
greedy-union-pool	1.042 167	1.193 238	1.757 868	0.024 327	0.975 673	0.266 567
greedy-upstream-window	1.031 489	1.227 970	1.710 767	-0.040 751	1.040 751	0.275 511
greedy-random	1.009 059	1.104 560	1.806 695	0.020 420	0.979 580	0.214 807
sa-random	0.998 808	1.149 864	1.750 100	-0.054 596	1.054 596	0.276 991
two-step-random	0.876 061	1.148 869	1.334 986	-0.128 479	1.128 479	0.318 905
two-step-union-pool	0.859 649	1.045 729	1.405 060	-0.057 920	1.057 920	0.401 216
greedy-violation-path	0.829 324	1.032 678	1.300 464	-0.048 521	1.048 521	0.277 805
greedy-critical-endpoint	0.746 583	0.923 081	1.188 662	-0.048 491	1.048 491	0.375 666

4.4 Difference Between Random and (Sampled) Union Pool on ISA and Tabu

The best strategy is tabu search with random candidate. As explained above, ISA with sampled union pool improves setup slack the most at the cost of area. However, tabu search with random candidate also improves setup slack significantly, while hold slack improves the most and decrease area consumption. Tabu search with random candidate gives the best tradeoff between slack and area.

4.4.1 Possible Reason

The advantage of ISA and tabu search is that they are designed to jump out of local optimum. Therefore, with random candidate policy, they are more likely to explore other possible solution.

5 Conclusion

In this work, we formulate timing fixing by useful skew as a clock-tree optimization problem over three reversible operations: buffer insertion, inserted buffer removal, and buffer resizing. These operations allow the optimizer to change clock arrival times at flip-flops and improve setup or hold slack without modifying the data-path logic. Since each edit may affect multiple paths and also changes area, the main challenge is not only timing repair, but the balance among setup slack, hold slack, and area cost.

Our experiments show that both candidate generation and acceptance strategy have a significant impact on the final result. Greedy search benefits from a larger candidate pool because it only accepts immediately improving moves. In contrast, simulated annealing and tabu search can explore beyond local optima, which makes random candidate generation more useful. Among all tested optimizers, tabu search with random candidates achieves the best average score. It does not produce the highest setup component, but it achieves the strongest hold improvement while also reducing area, giving the best overall trade-off. SA and ISA can aggressively improve timing, especially setup slack, but their area penalty reduces the final score. Two-step optimization follows a reasonable timing-first and area-recovery idea, but in our experiments it does not recover enough hold slack and area to compete with tabu search.

For future work, an adaptive strategy could select candidate policies according to the current violation pattern instead of using a fixed policy throughout the run. Another promising direction is to keep the exploration strength of tabu search while adding a stronger area-aware resize and removal polish in the late stage. This may preserve the timing gains from useful skew while further reducing unnecessary buffer area.

Author Contributions

Generative AI Usage Disclosure

We used generative AI tools during the development of this project. In particular, OpenAI Codex and Cursor was used extensively to assist with source-code implementation, including parser construction, clock-tree data structures, timing evaluation routines, optimization-loop implementation, test scripts, and refactoring. Approximately 80% of the submitted source code was initially generated or modified with the assistance of Codex or Cursor.

The algorithmic ideas, including the useful-skew optimization strategy, candidate generation policies, acceptance strategies, timing-area trade-off considerations, experimental design, and final optimizer selection, were designed and

evaluated by the team members. All AI-generated code was manually reviewed, integrated, debugged, and tested by the team. The reported experimental results were obtained by executing our implemented solver on the test cases; no experimental data or performance numbers were generated by AI tools.

We also used ChatGPT/Codex to assist with report organization, wording refinement, and documentation polishing. The final technical content, claims, results, and conclusions were checked and approved by the team.

References